

System Zarządzania Konferencjami (ConferenceDB)

Autor: Oliwia Kołacz

Indeks: 83099

Przedmiot: Zaawansowane Technologie Bazodanowe

Technologia: MS SQL Server 2022 (Docker) / Python 3 / Streamlit / pyodbc

Opis

System zarządzania konferencjami. Baza danych w MS SQL Server 2022 z aplikacją webową w Pythonie (Streamlit).

Uruchomienie

Baza danych

Skrypty SQL uruchamiamy w kolejności: 01_create_database.sql, 02_data.sql, 03_views.sql, 04_procedures.sql, 05_triggers.sql.

Aplikacja

```
cd app
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
nano .env # ustaw hasło do MS SQL
streamlit run streamlit_app.py
```

Aplikacja dostępna pod <http://localhost:8501>.

Wymagania projektu

#	Co?	Jak?
1	Tabele	W projekcie jest 10 tabel: Conferences, Tracks, Rooms, Speakers, Sessions, Attendees, TicketTypes, Tickets, SessionRegistrations, PriceHistory.
2	Rekordy	Każda tabela ma co najmniej 10 rekordów (np. 10 konferencji, 15 uczestników, 12 prelegentów, 14 sesji, 19 biletów, 27 zapisów na sesje).
3	PRIMARY KEY	Każda tabela ma klucz główny na kolumnie ID.
4	FOREIGN KEY	Zawiera 11 kluczy obcych.
5	UNIQUE	Zawiera 11 ograniczeń unikalności na kolumnach.
6	CHECK	Zawiera 16 ograniczeń warunkowych.

#	Co?	Jak?
7	DEFAULT	Zawiera 11 wartości domyślnych.
8	Widoki	Zawiera dwa widoki: vw_SessionSchedule (pokazuje pełne info o sesji w jednej tabeli) oraz vw_PopularSessions (popularność sesji jako procent zapełnienia sali względem pojemności).
9	Procedury składowane z parametrami	Zawiera dwie procedury: usp_PurchaseTicket (waliduje okno sprzedaży, dostępność biletów i status konferencji) oraz usp_RegisterForSession (waliduje istnienie sesji, ważność biletu, brak duplikatu zapisu).
10	Triggery	Zawiera dwa triggery: trg_AutoWaitlist (AFTER INSERT na SessionRegistrations) - sprawdza pojemność sali względem liczby zapisów Confirmed i nadaje status Waitlist gdy sala pełna. trg_PriceHistoryAudit (AFTER UPDATE na TicketTypes) - loguje każdą zmianę ceny do tabeli PriceHistory używając tabel inserted i deleted.
12	Aplikacja Python obiektowa	Aplikacja Streamlit w katalogu app/ z architekturą trójwarstwową. Warstwa danych: repozytoria DAO w db/repositories.py (ConferenceRepository, AttendeeRepository, SessionRepository itd.) z metodami list_all, get, create, update, delete. Warstwa logiki: serwisy w services.py (TicketService, RegistrationService, AdminService). Warstwa prezentacji: streamlit_app.py + 4 podstrony w pages/. Modele danych: dataclass w models.py.
13	Transakcje	W procedurach zastosowano BEGIN TRANSACTION / COMMIT TRANSACTION / ROLLBACK TRANSACTION w bloku BEGIN TRY ... BEGIN CATCH.